

## ИСПОЛЬЗОВАНИЕ ДОВЕРИТЕЛЬНОГО ИНТЕРВАЛА ПРИ ВЫПОЛНЕНИИ ИССЛЕДОВАНИЯ ПАРАЛЛЕЛЬНЫХ АЛГОРИТМОВ

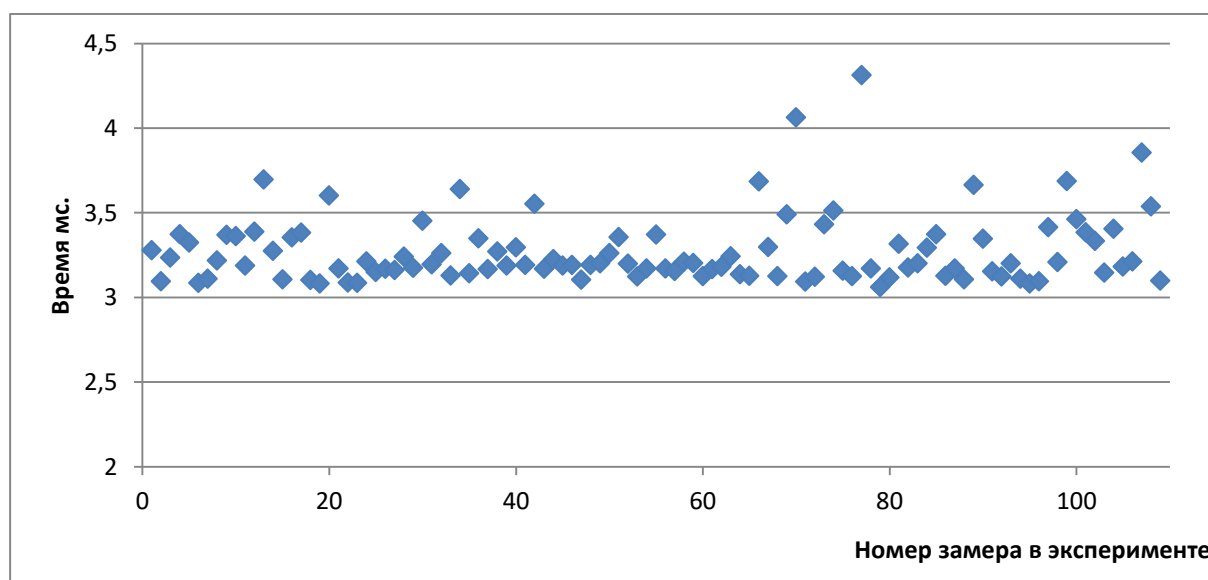
**Доверительный интервал** – вычисленный на основе выборки интервал значений признака, который с известной вероятностью содержит оцениваемый параметр генеральной совокупности.

Смысл вычисления доверительного интервала заключается в построении по данным выборки такого интервала, чтобы можно было утверждать с заданной вероятностью, что значение оцениваемого параметра находится в этом интервале. Другими словами, доверительный интервал с определенной вероятностью содержит неизвестное значение оцениваемой величины. Чем шире интервал, тем выше неточность.

В случае проведения экспериментального исследования параллельных алгоритмов использование доверительного интервала позволит сократить влияния выбросов на результат.

Так, в случае когда при замере времени работы алгоритма процессорные ядра были загружены задачами прикладных программ или операционной системы то в результате замера могут появляться значения которые сильно отличаются от стальных значений «выделяются» (так называемые выбросы). Выбросы могут быть выражены как в большую сторону (увеличенное время выполнения), так и в меньшую сторону (например, когда данные поместились в кеш и в последующем брались не из оперативной памяти).

На рисунке показан пример визуального отображения значения времени полученный при 110 замерах при выполнении алгоритма.



Среднеарифметическое значение времени работы алгоритма для полной выборки (110 замеров): **3,2716 мс.**

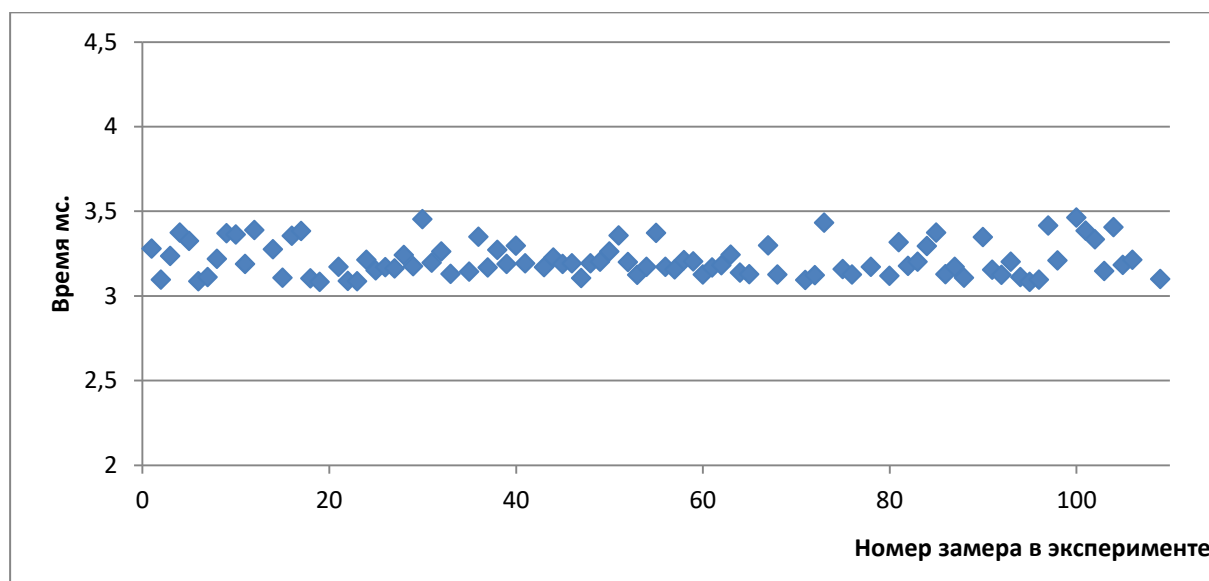
Самая простая форма доверительного интервала использует среднеарифметическое значение  $[X_{avg}]$  и стандартное (среднеквадратичное) отклонение  $[SD]$ :

$$SD = \sqrt{\frac{1}{N-1} \times \sum_{i=1}^N (X_i - X_{avg})^2}$$

Для сокращения влияния выбросов на результат из выборки убираются значения, которые не удовлетворяют условию:

$$X_{avg} - SD \leq X_i \leq X_{avg} + SD$$

В используемом тестовом наборе из **110** замеров **14** не подпали под условие. При визуализации значений выборка данных подходящих под условие доверительного интервала стала выглядеть следующим образом:

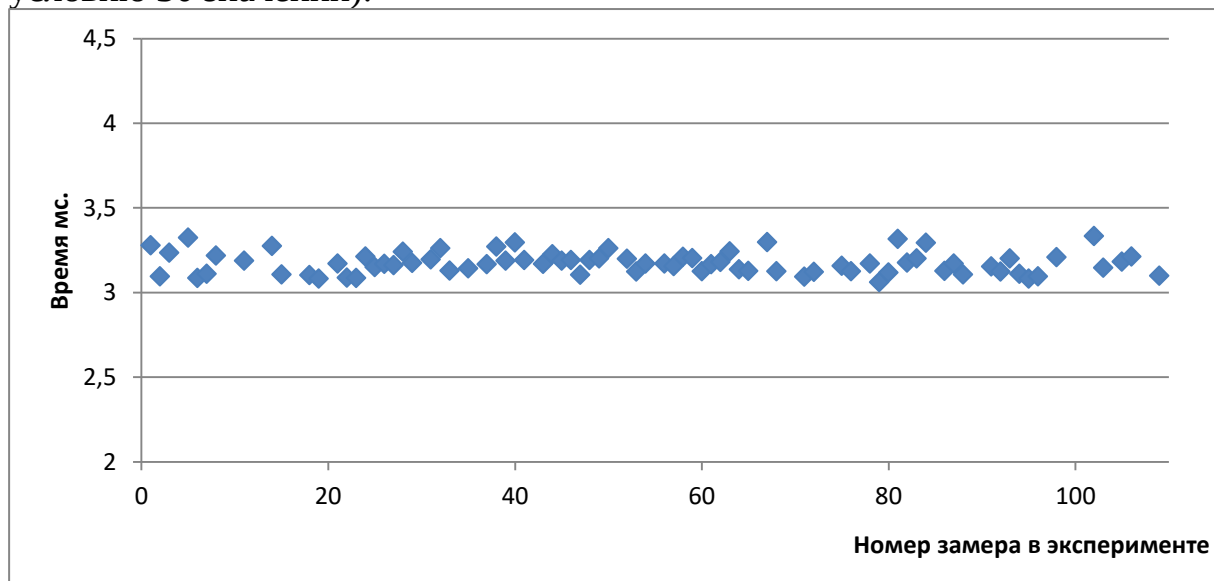


Для **96** элементов, которые удовлетворяют условию, рассчитывается новое среднеарифметическое значение: **3,2013 мс.** Как видно новое среднеарифметическое значение стало более точно описывать результат экспериментального исследования.

Помимо использования представленного условия можно применять условие с медианным значением по выборке:

$$X_{med} - SD \leq X_i \leq X_{med} + SD$$

В таком случае результат будет следующим (из **110** не подошли по условию **30** значений):



Среднеарифметическое значение по доверенному интервалу на основе медианы: **3,1738 мс.**

Как можно заметить существует разница в результатах. Наиболее оптимальным будет использование медианы, однако минимально достаточным в рамках лабораторных работ будет использование условия на основе среднеарифметического значения.

### Исходные коды для двух вариантов расчета

#### Вариант 1 условие $X_{avg} - SD \leq X_i \leq X_{avg} + SD$

```
// Функция расчёта среднеарифметического значения в доверительном интервале
// на основе среднеарифметического значения
// times - динамический одномерный массив с временем выполнения алгоритма
// cnt - количество элементов в массиве (выполненных экспериментов)
double AvgTrustedIntervalAVG(double*& times, int cnt)
{
    // вычисление среднеарифметического значения
    double avg = 0;
    for (int i = 0; i < cnt; i++)
    {
        // подсчет в переменную суммы
        avg += times[i];
    }
    // деление на количество
    avg /= cnt;

    // подсчет стандартного отклонения
    double sd = 0, newAVG = 0;
    int newCnt = 0;
    for (int i = 0; i < cnt; i++)
    {
        sd += (times[i] - avg) * (times[i] - avg);
    }
}
```

```

    }
    sd /= (cnt - 1.0);
    sd = sqrt(sd);
    // вычисление нового среднего значения в доверительном интервале
    // с использованием среднеарифметического значения
    //
    for (int i = 0; i < cnt; i++)
    {
        if (avg - sd <= times[i] && times[i] <= avg + sd)
        {
            newAVg += times[i];
            newCnt++;
        }
    }
    if (newCnt == 0) newCnt = 1;
    return newAVg / newCnt;
}

```

## Вариант 2 условие $X_{med} - SD \leq X_i \leq X_{med} + SD$

// Функция сортировки массива (сортировка пузырьком)

```

void bubbleSort(double *mas, int cnt) {
    bool exchanges;
    do {
        exchanges = false;
        for (int i = 0; i < cnt - 1; i++) {
            if (mas[i] > mas[i + 1]) {
                double temp = mas[i];
                mas[i] = mas[i + 1];
                mas[i + 1] = temp;
                exchanges = true;
            }
        }
    } while (exchanges);
}

```

// Функция расчёта среднеарифметического значения в доверительном интервале  
// на основе **медианного значения**  
// times - динамический одномерный массив с временем выполнения алгоритма  
// cnt - количество элементов в массиве (выполненных экспериментов)

```

double AvgTrustedIntervalMed(double*& times, int cnt)
{
    // вычисление среднеарифметического значения
    double avg = 0;
    for (int i = 0; i < cnt; i++)
    {
        // подсчет в переменную суммы
        avg += times[i];
    }
    // деление на количество
    avg /= cnt;

    // сортировка массива для получение медианы
    bubbleSort(times, cnt);
    // получение медианы
    double med = times[cnt / 2];

    // подсчет стандартного отклонения
    double sd = 0, newAVg = 0;
}

```

```

int newCnt = 0;
for (int i = 0; i < cnt; i++)
{
    sd += (times[i] - avg) * (times[i] - avg);
}
sd /= (cnt - 1.0);
sd = sqrt(sd);
// вычисление нового среднего значения в доверительном интервале
// с использованием медианного значения
for (int i = 0; i < cnt; i++)
{
    if (med - sd <= times[i] && times[i] <= med + sd)
    {
        newAVg += times[i];
        newCnt++;
    }
}
if (newCnt == 0) newCnt = 1;
return newAVg / newCnt;
}

```